

Klasyfikator flag bitowych — obie zdiagnozowane przyczyny naprawione

Odpowiedź na pytanie zostawione otwarte 6 sierpnia: czy naprawiać klasyfikator przed fine-tuningiem, czy uczyć na danych z ~44% błędnych etykiet. W trakcie prac **obalono własny wcześniejszy wynik** — patrz sekcja o weryfikacji.

PUNKT WYJŚCIA — ZGŁOSZENIE Z 6 SIERPNIA

Klasyfikator na szerszym korpusie: **Recall 55,9%, trafność maski 15,8%**. Zdiagnozowano dwie przyczyny: **(a)** za krótkie okno obserwacji, **(b)** bajty mieszane — flaga i skalar częściowy w tym samym bajcie. **Obie zostały naprawione** — pierwsza w pełni, druga częściowo (patrz weryfikacja).

PRZYCZYNA (A) — ZA KRÓTKIE OKNO OBSERWACJI

6 SIERPNIA Recall 55,9% obserwacja epizodyczna, pojedyncze uruchomienie	EKSPERYMENT 4.5 / 4.6 Recall 85,0% ciągła obserwacja + strojenie progów, Precision 100%
EKSPERYMENTY 4.7-4.10 Recall 100% klasyfikator uczony, Precision 97,3%	

Zastrzeżenie: to nie są te same korpusy, więc nie jest to czysty ciąg jednej metryki. Wspólne jest to, że każdy kolejny krok mierzono na trudniejszych, nie łatwiejszych danych.

PRZYCZYNA (B) — BAJTY MIESZANE · NAJMOCNIEJSZY WYNIK SESJI

Problem był **źle rozumiany**. Maski `seen0 & seen1` nie jest „ogólnie kiepska” — jest idealna dla bajtów z samymi flagami i ma **dokładnie zero** trafień tam, gdzie flaga dzieli bajt ze skalem.

Trafność maski dokładnej · test na prawdziwym MCP2515, 8528 bitów

METODA	BAJTY CZYSTE (~78%)	BAJTY MIESZANE (~22%)	RAZEM
maska <code>seen0 & seen1</code>	137/137 = 100%	0/30 = 0%	82%
sieć per bit (korpus ciągły)	137/137 = 100%	30/30 = 100%	100%

Uwaga: ten wynik został skorygowany — patrz następna sekcja. Gdyby raportować samą średnią, wyszłoby 82% i wyglądałoby to na drobną niedoskonałość — zamiast na całkowitą bezradność w co piątym przypadku. **Dlatego ewaluacja rozбивa oba typy osobno.**

WERYFIKACJA WŁASNEGO WYNIKU – I JEGO KOREKTA

Wynik 100% wymagał sprawdzenia: czy sieć nauczyła się **prawdziwej zasady**, czy **regularności generatora**? W korpusie flagi siedziały zawsze na najniższych bitach, a skalar tworzył ciągły zakres nad nimi. Bity rozproszono i powtórzono pomiar na tym samym sprzęcie.

Trafność maski w bajtach mieszanych · test na prawdziwym MCP2515, 8432 bity

WARIANT TESTU	MODEL	MASKA MIESZANE
bity ciągłe (pierwotny korpus)	uczony na ciągłych	30/30 = 100%
bity rozproszone	ten sam model	5/37 = 14%
bity rozproszone	uczony na reprezentatywnych	22/37 = 59%

Model nie generalizował – wynik 100% był zawyżony

Sieć nauczyła się w znacznej mierze **układu pozycji bitów**, a nie zasady o sprzężeniu statystycznym. Po usunięciu regularności generatora trafność maski spada ze 100% do 14%. Po przeuczeniu na danych reprezentatywnych: 59% – wciąż dużo wobec zera, ale **to nie jest problem rozwiązany**.
Zastrzeżenie o ciągłych zakresach bitów było zapisane pod wynikiem 4.11 jako ryzyko teoretyczne. **Sprawdzenie zajęło jedno popołudnie i obaliło główną liczbę tamtego eksperymentu.**

Uczciwy bilans na trudniejszym, rozproszonym korpusie

METRYKA	MASKA SEEN0 & SEEN1	SIEĆ PER BIT
Precyzja per bit	26,8%	96,5%
F1 per bit	42,3%	98,2%
Maska w bajtach mieszanych	0%	59%

DLACZEGO SIEĆ, A NIE PROSTA REGUŁA

Ten sam protokół: próg dobierany na uczących, oceniany na sprzęcie

PODEJŚCIE	F1	MASKA MIESZANE
najlepszy pojedynczy próg	57,8%	10%
najlepsza para cech	81,9%	57%
sieć (11 cech, 1105 par.)	99,9%	100%

Kontrapunkt: gdzie sieć NIE była potrzebna

W Eksperymentcie 4.7 ta sama analiza wskazała **regułę**: zmiana jednej stałej (limit bitów 6 → 7) dała +5 pp F1 przy zerowym koszcie obliczeniowym – prawie tyle co sieć.
To różnica między „użyłem ML” a „sprawdziłem, kiedy ML jest potrzebny”.

CO SIĘ NIE UDAŁO – ZGŁOSZONE SAMODZIELNIE

Trzy próby testu odporności – hipoteza ODRZUCONA

PRÓBA	ZMIENNA	ZMIENIŁA STATYSTYKI?
4.8	prędkość magistrali	nie – błąd projektowy
4.9	liczba CAN ID	nie – błąd projektowy
4.10	skala okresów ramek	tak, 15×

Dwie pierwsze manipulowały parametrami **poziomu magistrali**; statystyki per-bajt zależą od **stosunku próbkowania do dynamiki sygnału**. Przy działającej manipulacji: wszystkie metody stabilne (rozstępy 1,2-2,5 pp), **najstabilniejsza okazała się reguła**.

Wycofany wniosek

Na teście obejmującym 132 pozycje wyszło, że reguła bije sieć – i tak to zapisano. Przy próbie **22× większej** kierunku się odwrócił. Różnica opierała się na **dwóch pozycjach**, czyli na szumie.

Wniosek metodologiczny: przy 20 pozytywach w zbiorze testowym różnice rzędu kilku punktów procentowych **nie są interpretowalne**.

WĄTEK PLATFORMY – WYNIK UBOCZNY, ALE MOCNY

Eksperyment 4.5 powtórzono na **drugiej, niezależnej platformie** (Orange Pi Zero 3: inny procesor, architektura, system i magistrala SPI). Wynik **identyczny co do liczby**: Recall 85,0%, Precision 100%.

WNIOSEK

Własność METODY, nie płytki

dwie platformy, ten sam wynik

PARAMETRÓW SIECI

1105

wdrożenie bez PyTorch

UCZENIE NA PŁYTCIE

68 s

inferencja 282 μs

DANYCH ZE SPRZĘTU

57 280

bitów, 3 niezależne zbiory

NAJWIĘKSZE OGRANICZENIE – PODANE WPROST

Cały ruch jest syntetyczny

Test na innych ziarnach ogranicza ryzyko zapamiętywania, ale inne ziarno to wciąż ten sam generator. **Wyniki są mocną przesłanką, nie dowodem** na prawdziwej magistrali.

Zastrzeżenie o ciągłych zakresach bitów **zostało sprawdzone i potwierdziło się** (Eksperyment 4.12) – co podnosi wiarygodność pozostałych zastrzeżeń, bo pokazuje, że nie są kurtuazyjne.

1. **Etap D (fine-tuning) jest odblokowany** — klasyfikator-nauczyciel nie jest już wąskim gardłem. Ruszać, czy najpierw walidacja na prawdziwej maszynie?
 2. **Sieć jako domyślny klasyfikator**, czy reguła z poprawioną stałą (+5 pp, zero kosztu) i sieć jako opcja?
 3. **Druga platforma sprzętowa** w zakresie pracy, czy rozmywa tezę o mikrokontrolerze?
 4. **Walidacja na maszynie rolniczej** — osobny eksperyment w tej pracy, czy materiał na kolejną?
-

Zakres pomiarów: ~7600 pozycji bajtowych i 57 280 bitów z prawdziwego kontrolera MCP2515, w trzech niezależnych zbiorach. Protokół: uczenie i walidacja na wirtualnych magistralach SocketCAN, test wyłącznie na sprzęcie.

Źródła: `Eksperyment_4.6_...`, `Eksperyment_4.7-4.10_...`, `Eksperyment_4.11_Maska_Bitowa_20260815.md` ·
repozytorium GitHub, commit `5b9e2ff`